

Aula 4 (Módulo 01) — SSH

1. O que é SSH (visão simples)

SSH (*Secure Shell*) é o protocolo padrão para acessar um terminal de um computador remoto pela rede, de forma **criptografada** (ninguém no meio do caminho consegue ler o que você digita ou o que o servidor responde). É como o "controle remoto" mais usado no mundo Linux/servidor: a imensa maioria dos servidores na internet é administrada via SSH.

```
ssh usuario@endereco_do_servidor
```

Por que importa para segurança: SSH é, ao mesmo tempo, a ferramenta de administração mais usada por administradores legítimos e um dos alvos mais atacados de qualquer servidor exposto à internet. Se você já colocou qualquer servidor online, provavelmente viu (ou vai ver) milhares de tentativas de login SSH por dia vindas de bots pela internet inteira tentando senhas comuns — isso se chama **brute force** (tentativa massiva de senhas).

2. Autenticação: senha vs. chave pública/privada

Por senha

Mais simples, mas mais fraco: qualquer um que descubra a senha (por brute force, vazamento, phishing) consegue entrar.

Por par de chaves (recomendado)

Você gera duas "chaves" matematicamente relacionadas:

- **Chave privada:** fica só no seu computador, **nunca é compartilhada**. É como a chave física da sua casa.
- **Chave pública:** você copia para o servidor (dentro de `~/.ssh/authorized_keys`, na conta do usuário remoto). É como uma fechadura que só a sua chave privada específica consegue abrir.

```
ssh-keygen -t ed25519          # gera o par de chaves (ed25519 é o algoritmo recomendado hoje)
ssh-copy-id usuario@servidor    # copia sua chave pública para o servidor
```

Depois disso, `ssh usuario@servidor` não pede mais senha — usa a chave.

Por que importa para segurança: autenticação por chave é consideravelmente mais resistente a brute force do que senha (a chave privada tem um tamanho e complexidade matemática impraticável de adivinhar por tentativa). Por isso é comum, em ambientes bem configurados, **desabilitar completamente login por senha via SSH** e aceitar só chaves — isso é feito no arquivo de configuração do servidor, `/etc/ssh/sshd_config`, com a diretiva `PasswordAuthentication no`.

3. Onde ficam as evidências de SSH

- **No servidor**, cada tentativa de conexão (bem-sucedida ou não) é registrada em log — em Ubuntu/Debian normalmente `/var/log/auth.log`, em RHEL/Fedora `/var/log/secure` (os mesmos arquivos que mencionamos na Aula 1 deste módulo, a propósito de `sudo`).
- Uma linha típica de **falha** de login por senha se parece com: `Failed password for usuario invalido from 203.0.113.45 port 51234 ssh2`
- Uma linha típica de **sucesso**: `Accepted publickey for nicolas from 198.51.100.10 port 52211 ssh2`

Por que importa para segurança: essas duas linhas, sozinhas, já dão para responder as perguntas centrais de uma investigação: **quem** (usuário), **de onde** (IP de origem), **quando** (timestamp do log), **como** (senha ou chave), **conseguiu ou não**. Muitas regras de detecção de SOC (que vamos formalizar no Módulo 09 - SIEM) são literalmente: "mais de N tentativas de `Failed password` do mesmo IP em M minutos" → possível brute force → alerta.

4. Boas práticas (ligando com o que já vimos)

- **Least privilege** (Módulo 00, Aula 2): usuário SSH não deveria, por padrão, logar direto como `root` — a diretiva `PermitRootLogin` no `/etc/ssh/sshd_config` bloqueia login direto de root via SSH, forçando o uso de um usuário comum + `sudo` (que fica logado, como vimos na Aula 2 deste módulo).
- **Chave em vez de senha**: já explicado acima.
- **Fail2ban** (fora do escopo de hoje, mas vale mencionar): uma ferramenta comum que lê o log de SSH automaticamente e **bloqueia temporariamente o IP** que errou a senha várias vezes — uma resposta automatizada a um padrão de ataque, algo que vamos formalizar quando chegarmos a Detection Engineering e resposta automatizada.

5. Exercício prático

Se você tiver acesso a qualquer máquina Linux com log de SSH disponível (mesmo que seja só a sua própria, testando localmente):

```
sudo grep "sshd" /var/log/auth.log 2>/dev/null | tail -20 \  
|| sudo grep "sshd" /var/log/secure 2>/dev/null | tail -20
```

Se o SSH nunca foi usado nessa máquina, o resultado pode vir vazio — isso também é uma informação válida (linha de base sem atividade). Se aparecer alguma linha, tente identificar: foi sucesso ou falha? Qual usuário? Qual IP de origem? (Não se preocupe com o comando `grep` em si agora — vamos estudá-lo a fundo, junto com `awk` e `sed`, na aula de comandos de investigação mais adiante neste módulo.)

6. Recapitulando

- SSH = acesso remoto criptografado a um terminal; onipresente em servidores Linux.
- Autenticação por **chave pública/privada** é mais segura que por senha; chave privada nunca sai do seu computador.
- Logs de SSH (`/var/log/auth.log` ou `/var/log/secure`) registram quem, de onde, como, sucesso ou falha — base de detecção de brute force.

- `PermitRootLogin no` e `PasswordAuthentication no` são configurações de hardening comuns em `/etc/ssh/sshd_config`.

Próxima aula

Logs no Linux: `syslog` e `journalctl` — como o sistema centraliza mensagens de log de diferentes serviços (incluindo o próprio SSH) e como consultá-los de forma eficiente.

Fontes recomendadas

- **OpenSSH — documentação oficial**: referência completa do protocolo, `ssh_config` e `sshd_config`.
<https://www.openssh.com/manual.html>
- **MITRE ATT&CK — T1110 (Brute Force)**: técnica oficial associada a ataques de força bruta, incluindo contra SSH.
<https://attack.mitre.org/techniques/T1110/>
- **CISA — "Alert on SSH Brute Force"** (boas práticas de hardening de SSH em ambientes expostos): consulte sempre a página oficial da CISA ([cisa.gov](https://www.cisa.gov)) para orientações atualizadas de hardening.